机器学习基础作业4

宋思逸

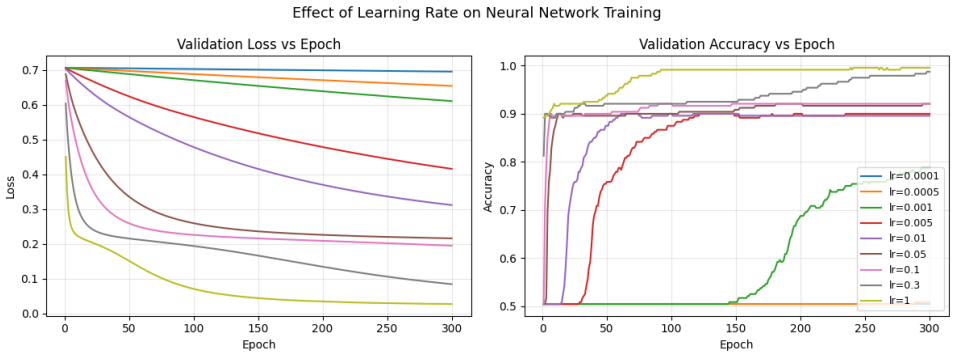
2400010816

2026 年 4 月 2 日

这是一个代码实践题。我们先废话一下，最后附上代码，并在代码中附上注释。

我们希望控制数据、网络结构和初始条件等各种条件都相同的情况下，仅改变学习率，考察训练效果的变化。

我们先来从图片看结果。



从图片当中容易看出，学习率较小时，损失下降慢，收敛轮次晚，训练效率低；当学习率中等时，收敛更快且最终验证精度更高；学习率过大则可能导致曲线波动，验证损失不稳定，泛化性能下降。

下面简要叙述代码思路

1构造一个适合神经网络分类的非线性数据集；

2建立固定结构的两层神经网络；

3选取多组不同的学习率进行对照实验；

4保证其他条件一致，只改变学习率；

5记录并比较损失、准确率和收敛速度；

6根据实验结果分析学习率对训练稳定性、收敛速度和最终性能的影响。

```python
import math
import time
from dataclasses import dataclass
from typing import Dict, List, Tuple

import numpy as np
#这里在做准备工作


def set_seed(seed: int = 42) -> None:
        np.random.seed(seed)
#固定随机性，即随机生成但是结果可以复现


def make_two_moons(n_samples: int = 1200, noise: float = 0.15) -> Tuple[np.ndarray, np.ndarray]:
#这里我们手动构造二分类数据
        n_half = n_samples // 2
```

```
        t1 = np.random.rand(n_half) * math.pi
        x1 = np.c_[np.cos(t1), np.sin(t1)]
#这类数据是半圆弧，通过(cost,sint)构造
        t2 = np.random.rand(n_samples - n_half) * math.pi
        x2 = np.c_[1.0 - np.cos(t2), 1.0 - np.sin(t2) - 0.5]
#另一条半圆弧
        x = np.vstack([x1, x2])
        y = np.hstack([np.zeros(n_half, dtype=np.int64), np.ones(n_samples - n_half,
        ↪  dtype=np.int64)])
#将两类数据合并
        x += np.random.randn(*x.shape) * noise
        return x, y
#添加噪声

def train_val_split(x: np.ndarray, y: np.ndarray, val_ratio: float = 0.2) -> Tuple[np.ndarray,
↪  np.ndarray, np.ndarray, np.ndarray]:#这里在划分数据集和训练集，打乱索引后按0.8训练0.2测试
        idx = np.arange(len(x))
        np.random.shuffle(idx)
        n_val = int(len(x) * val_ratio)
        val_idx = idx[:n_val]
        train_idx = idx[n_val:]
        return x[train_idx], y[train_idx], x[val_idx], y[val_idx]


def one_hot(y: np.ndarray, n_classes: int) -> np.ndarray:
        out = np.zeros((len(y), n_classes), dtype=np.float64)
        out[np.arange(len(y)), y] = 1.0
        return out


def softmax(z: np.ndarray) -> np.ndarray:#先减去每行最大值再指数化，避免溢出
        z = z - np.max(z, axis=1, keepdims=True)
        exp_z = np.exp(z)
        return exp_z / np.sum(exp_z, axis=1, keepdims=True)


def cross_entropy(probs: np.ndarray, y: np.ndarray) -> float:#取真类，对应概率后求出log(p)的均值，
↪  注意这里eps是为了防止出现log(0)
        eps = 1e-12
        p = probs[np.arange(len(y)), y]
        return -float(np.mean(np.log(p + eps)))


@dataclass
class RunStats:#将数据结果打包
        lr: float
        train_loss: List[float]
        val_loss: List[float]
```

```
        train_acc: List[float]
        val_acc: List[float]
        converged_epoch: int
        final_val_acc: float
        final_val_loss: float


class MLP:#这里有输入2维到隐藏层，激活函数和隐藏层到输出2维
        def __init__(self, in_dim: int, hidden_dim: int, out_dim: int, init_scale: float = 0.2):
                self.w1 = np.random.randn(in_dim, hidden_dim) * init_scale
                self.b1 = np.zeros((1, hidden_dim))
                self.w2 = np.random.randn(hidden_dim, out_dim) * init_scale
                self.b2 = np.zeros((1, out_dim))

def forward(self, x: np.ndarray) -> Tuple[np.ndarray, Dict[str, np.ndarray]]:#前向传播
        z1 = x @ self.w1 + self.b1
        a1 = np.maximum(z1, 0.0)
        logits = a1 @ self.w2 + self.b2
        probs = softmax(logits)
        cache = {"x": x, "z1": z1, "a1": a1, "probs": probs}
        return probs, cache

def backward(self, cache: Dict[str, np.ndarray], y: np.ndarray) -> Dict[str, np.ndarray]:#反向传
↪  播
        x = cache["x"]
        z1 = cache["z1"]
        a1 = cache["a1"]
        probs = cache["probs"]
        n = x.shape[0]

dlogits = probs.copy()
dlogits[np.arange(n), y] -= 1.0
dlogits /= n

dw2 = a1.T @ dlogits
db2 = np.sum(dlogits, axis=0, keepdims=True)
da1 = dlogits @ self.w2.T
dz1 = da1 * (z1 > 0)
dw1 = x.T @ dz1
db1 = np.sum(dz1, axis=0, keepdims=True)

return {"dw1": dw1, "db1": db1, "dw2": dw2, "db2": db2}

def step(self, grads: Dict[str, np.ndarray], lr: float) -> None:
        self.w1 -= lr * grads["dw1"]
        self.b1 -= lr * grads["db1"]
        self.w2 -= lr * grads["dw2"]
        self.b2 -= lr * grads["db2"]
```

```
110
111
112 def accuracy(probs: np.ndarray, y: np.ndarray) -> float:
113         pred = np.argmax(probs, axis=1)
114         return float(np.mean(pred == y))
115
116
117 def detect_convergence(loss_curve: List[float], window: int = 20, threshold: float = 1e-4) ->
    ↪  int:#取最近window验证损失，若极差小于阈值，则认为基本收敛
118         if len(loss_curve) < window + 1:
119                 return len(loss_curve)
120         for i in range(window, len(loss_curve)):
121                 recent = loss_curve[i - window : i]
122                 if max(recent) - min(recent) < threshold:
123                         return i + 1
124         return len(loss_curve)
125
126
127 def run_experiment(lrs: List[float],epochs: int = 300,hidden_dim: int = 32,seed: int =
    ↪  42,)->List[RunStats]:
128         set_seed(seed)
129         x, y = make_two_moons(n_samples=1200, noise=0.15)
130         x_train, y_train, x_val, y_val = train_val_split(x, y, val_ratio=0.2)
131 #这里，先生成一次固定数据并切分，保存当前随机状态，每测试一个学习率时，先恢复相同的随机状态，再初始化
    ↪  模型
132
133 #仅使用训练集计算mean和std，训练集和测试集都用同一组统计量变换
134         mean = x_train.mean(axis=0, keepdims=True)
135         std = x_train.std(axis=0, keepdims=True) + 1e-8
136         x_train = (x_train - mean) / std
137         x_val = (x_val - mean) / std
138
139         base_rng_state = np.random.get_state()
140         results: List[RunStats] = []
141
142         for lr in lrs:
143 #让每次学习都从同样的初始化状态开始
144                 np.random.set_state(base_rng_state)
145                 model = MLP(in_dim=2, hidden_dim=hidden_dim, out_dim=2, init_scale=0.2)
146
147                 train_loss_curve: List[float] = []
148                 val_loss_curve: List[float] = []
149                 train_acc_curve: List[float] = []
150                 val_acc_curve: List[float] = []
151
152                 for _ in range(epochs):
153                         train_probs, cache = model.forward(x_train)
154                         tr_loss = cross_entropy(train_probs, y_train)
```

```
                        tr_acc = accuracy(train_probs, y_train)

                        grads = model.backward(cache, y_train)
                        model.step(grads, lr=lr)

                        val_probs, _ = model.forward(x_val)
                        va_loss = cross_entropy(val_probs, y_val)
                        va_acc = accuracy(val_probs, y_val)

                        train_loss_curve.append(tr_loss)
                        val_loss_curve.append(va_loss)
                        train_acc_curve.append(tr_acc)
                        val_acc_curve.append(va_acc)

                conv_epoch = detect_convergence(val_loss_curve, window=20, threshold=1e-4)
                results.append(
                        RunStats(
                                lr=lr,
                                train_loss=train_loss_curve,
                                val_loss=val_loss_curve,
                                train_acc=train_acc_curve,
                                val_acc=val_acc_curve,
                                converged_epoch=conv_epoch,
                                final_val_acc=val_acc_curve[-1],
                                final_val_loss=val_loss_curve[-1],
                        )
                )

        return results


def print_summary(results: List[RunStats]) -> None:
        print("\n=== Learning Rate Comparison Summary ===")
        print(f"{'LR':>10} {'Final Val Loss':>15} {'Final Val Acc':>15} {'Converged
        ↪  Epoch':>18}")
        for r in results:
                print(f"{r.lr:>10.4g} {r.final_val_loss:>15.6f} {r.final_val_acc:>15.4f}
                ↪  {r.converged_epoch:>18}")

        best = max(results, key=lambda x: x.final_val_acc)
        print("\nBest validation accuracy:")
        print(f"LR={best.lr:.4g}, Val Acc={best.final_val_acc:.4f}, Val
        ↪  Loss={best.final_val_loss:.6f}")


def try_plot(results: List[RunStats]) -> None:
        try:
                import matplotlib.pyplot as plt
```

```
        except Exception:
                print("\nmatplotlib is not installed, skip plotting.")
                return

        epochs = np.arange(1, len(results[0].train_loss) + 1)
        fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))
#绘制图像：验证损失-epoch曲线；验证准确率-epoch曲线
#每条线对应一个学习率
#小学习率，下降慢，收敛慢
#合适学习率，下降快且稳定
#过大学习率，可能震荡甚至变差

        for r in results:
                axes[0].plot(epochs, r.val_loss, label=f"lr={r.lr:g}")
                axes[1].plot(epochs, r.val_acc, label=f"lr={r.lr:g}")

        axes[0].set_title("Validation Loss vs Epoch")
        axes[0].set_xlabel("Epoch")
        axes[0].set_ylabel("Loss")
        axes[0].grid(alpha=0.3)

        axes[1].set_title("Validation Accuracy vs Epoch")
        axes[1].set_xlabel("Epoch")
        axes[1].set_ylabel("Accuracy")
        axes[1].grid(alpha=0.3)

        axes[1].legend(loc="lower right", fontsize=9)
        fig.suptitle("Effect of Learning Rate on Neural Network Training", fontsize=13)
        fig.tight_layout()
        plt.show()


def main() -> None:
        lrs = [1e-4, 5e-4, 1e-3, 5e-3, 1e-2, 5e-2, 1e-1, 3e-1, 1.0]
        start = time.time()
        results = run_experiment(lrs=lrs, epochs=300, hidden_dim=32, seed=42)
        print_summary(results)
        try_plot(results)
        print(f"\nDone in {time.time() - start:.2f}s")


if __name__ == "__main__":
        main()
```